How Robust Are Exception Handling Defect Models?
A2: Baseline, Artifacts, and Related Work

Group 9: Emmanuel Adegboye, Benjamin Emeshili, Marco Spensieri
Software Maintenance and Evolution, Ontario Tech University
Artifacts: https://github.com/Detola12/software_replication

***Abstract*—We are replicating de Pádua and Shang (MSR 2018), who found that some exception handling anti-patterns help explain post-release defects. This milestone reports a working baseline. Running the authors' replication package end to end reproduces their merged dataset exactly and reproduces the BASE and anti-pattern (BSAP) models in Table 4 of the paper. The exception flow (BSFC) model does not reproduce: a leftover line in the published code makes BSFC reuse the BSAP data, so every BSFC result is a copy of BSAP. The package also needed one fix to run on R 4.3 or later.**

I. Study Context and Related Work

De Pádua and Shang [1] asked whether exception handling practices add explanatory power to defect models built from traditional metrics. For each project they fit three logistic regression models of whether a file has a post-release defect: BASE (size, complexity, churn and ownership), BSFC (BASE plus exception flow characteristics, their RQ1) and BSAP (BASE plus 17 exception handling anti-patterns, their RQ2). The anti-pattern metrics come from their earlier study of anti-pattern prevalence [2], and the flow metrics from their exception flow analysis [3].

We study the same systems and releases as the original paper (Table I) and use the authors' published data. Keeping the data fixed means any difference we find comes from the analysis rather than from data collection.

TABLE I
Studied systems (from the original study)

Project	Lang.	Release	Last post-release tag	Files w/ catch
Hadoop	Java	release-2.6.0	rel/release-2.6.5	926
Hibernate	Java	5.0.0.Final	5.0.16	478
Umbraco	C#	release-7.6.0	release-7.6.12	321

The literature informs three choices. First, Jiarpakdee et al. [4] show that how correlated metrics are removed changes which metrics a defect model ranks as important, and Shepperd et al. [5] found that analysis choices explain much of the variation between published defect studies. Our extension therefore varies the five settings the original study fixed: the correlation cutoff (0.7), the redundancy cutoff ($R^2 = 0.9$), the predictor budget (one per 15 files), the $\log_{10}(x+1)$ transform and the backward-selection threshold ($p < 0.05$). Second, Tantithamthavorn et al. [6] recommend the optimism-reduced bootstrap for defect models, so we keep the original 1,000-repetition bootstrap and add a fixed seed. Third, following Shull et al. [7], a close replication on the original data separates reproducibility (do we get the same numbers?) from robustness (do the conclusions survive other reasonable choices?). The predictor budget comes from Harrell's guidance [8], which we revisit in the sensitivity analysis.

II. Data and Artifacts

Source and revision. The data and code are the authors' replication package.[1] We downloaded it on 2 October 2026 from the site's GitHub source (`guipadua/guipadua.github.io`, folder `resources/`) because the direct download link was blocked on our network. Both archives are stored unchanged in our repository under `original-package/`, with these SHA-256 prefixes:

- `.._scripts_with_input_data.zip`: `2d6387a9ee0bdbd2` (5 input CSVs, 2 R notebooks, 2 helper scripts, the authors' saved outputs)
- `.._data_without_missing.zip`: `6a8eee1f5e5940c8` (the authors' merged dataset)

Inputs. `File_Level-Understand.csv` holds product metrics from the Understand tool. `File_Level-Git-Jira.csv` holds process metrics and pre- and post-release defect counts. `File_Level-Catch_Based`, `Try_Based` and `Throws_Based.csv` hold the exception handling metrics produced by the authors' JTratch and NTratch analyzers.

Collection and cleaning. We did not re-collect data. Notebook 0 merges the five files into one table of 10,195 files and 159 columns and removes files with missing values (the authors' step MC1). Notebook 1 keeps files that contain catch blocks, leaving 890 Hadoop, 440 Hibernate and 230 Umbraco files for modeling.

Environment. R 4.6.1 (Windows 11 x64) in RStudio, with the rms, Hmisc, caret, e1071, plotly, BiodiversityR, logistf and rmarkdown packages installed from CRAN in October 2026. `sessionInfo()` output is saved per machine in `logs/`. No LLMs are used in the analysis.

[1] https://guipadua.github.io/eh-model-defects2018

TABLE II
ORIGINAL PAPER VS. OUR REPRODUCTION (PAPER / OURS)

Project	Model	Budget	Pot. pred.	Cutoff	Opt.-red. R^2
Hadoop	BASE	59 / 59	10 / 10	.70 / .70	.33 / .33
	BSAP	59 / 59	23 / 23	.70 / .70	.35 / .35
	BSFC	59 / 59	**31 / 23**	.70 / .70	**.37 / .35**
Hibernate	BASE	29 / 29	18 / 18	.70 / .70	.21 / .21
	BSAP	29 / 29	19 / 19	.70 / .70	.23 / .23
	BSFC	29 / 29	**27 / 19**	.70 / .70	**.28 / .23**
Umbraco	BASE	15 / 15	12 / 12	.70 / .70	.09 / .09
	BSAP	15 / 15	16 / 16	.67 / .67	.18 / .18
	BSFC	15 / 15	**23 / 16**	**.24 / .67**	**.09 / .18**

Bold marks values that differ from the paper.

Repository. The repository contains the original package, our working copy of the code with each change marked `# FIX`, the authors' saved outputs, our run's outputs, the reproducibility log, and a README with install and run steps.

III. WORKING BASELINE

We ran both notebooks end to end on the study data on 2 October 2026. The model notebook takes about 25 minutes. Table II compares our results with Table 4 of the paper.

Observations.

1) *Data preparation reproduces exactly.* Our merged dataset matches the authors' published one in all 10,195 rows and 159 columns.
2) *BASE and BSAP reproduce.* Predictor budgets, numbers of potential predictors, adjusted correlation cutoffs and optimism-reduced R^2 all match Table 4. Coefficients match the authors' saved output to about 10^{-12}.
3) *The significant metrics match Table 5 for BASE and BSAP.* The anti-patterns the paper reports are significant in our run too: Ignoring Interrupted Exception and Log and Throw in Hadoop, Dummy Handler in Hibernate, and Dummy Handler and Generic Catch in Umbraco.
4) *Bootstrap values vary slightly between runs.* Optimism-reduced values differ from the authors' saved output by at most 0.023, because the code sets no random seed.
5) *BSFC does not reproduce.* In our output and in the authors' own saved output, every BSFC row is identical to the BSAP row, including its formula and significant metrics. The cause is line 223 of `construction_functions.R`, `data_list = all_list_omitted_m2`, a leftover line that overwrites the function's input with the BSAP data. The published package therefore cannot produce the paper's BSFC results. [If the fixed rerun is done: add the corrected BSFC values here and say whether they match the paper. Otherwise keep: "We are rerunning BSFC with this line removed."]

Preparing the new study context. We located all five settings in the code. The correlation cutoff is the variable `CORR_THRESHOLD`. The predictor budget is the literal 15, hard-coded in four places across both helper files. The redundancy cutoff is never written; it comes from the default of Hmisc's `redun()`. The transform is `log10(x + 1)` in both helper files, and the selection threshold is `sls = .05` in `fastbw()`. The next step moves all five into one configuration file with a fixed seed, so each sensitivity configuration is a one-line change.

IV. SCOPE AND LIMITATIONS

The package does not run unmodified on current R. Since R 4.3, `&&` requires a single logical value. Line 270 of `analysis_functions.R` applies it to `is.na()` of a fitted model, which returns one value per model element, and stops with an error. We replaced `!is.na(refit)` with `!isTRUE(is.na(refit))`, which keeps the original behaviour. Anyone rerunning the package needs this fix; it is documented in the README and log.

The BSFC bug affects RQ1 of the original paper. Until the corrected BSFC model is verified, our sensitivity analysis can only be compared with the paper for BASE and BSAP. If the corrected BSFC still differs from Table 4, we will report the gap as a reproducibility finding and limit our RQ2 and RQ3 to BASE and BSAP, as planned in the proposal.

Small samples after cleaning. Removing files with missing data leaves fewer files than the paper's Table 1 reports (Umbraco: 230 modeled files against 321 with catch blocks). This follows the authors' procedure, but with only 15 predictors allowed for Umbraco, a stricter budget of one per 20 files leaves 11 predictors and may make some Umbraco configurations impossible to fit. We will report those as not estimable.

Scattered settings. Because the budget appears in four places and the redundancy cutoff relies on a default, changing one copy and not the others would silently produce a mixed configuration. Centralizing the settings comes before any sensitivity run.

Changes to the proposal. The scope is unchanged. Two items move from A2 to A3: the configuration file with a fixed seed, and the corrected BSFC rerun if it is not finished for this milestone. The midterm on 30 October may delay the Umbraco runs by about a week; A3 (6 November) will cover Hadoop and Hibernate first.

V. PROGRESS AND CONTRIBUTIONS

Completed: the authors' package archived with checksums; the full pipeline run end to end on R 4.6.1; data preparation, BASE and BSAP reproduced and compared with Tables 4 and 5; the R 4.3 incompatibility fixed; the BSFC bug diagnosed; all five sensitivity settings located in the code; and a repository with the code, data, outputs, reproducibility log and README (commit `e4c6a4f`). Table III lists each member's tasks.

TABLE III
MEMBER TASKS AND CONTRIBUTIONS

Member	Main tasks and outputs
E. Adegboye	First end-to-end run and comparison with the paper; R 4.3 fix; BSFC bug diagnosis; repository and reproducibility log; Sections I and V
B. Emeshili	[Benjamin: what you did, e.g., independent rerun on own machine, BSFC fix, Section III]
M. Spensieri	[Marco: what you did, e.g., environment check on own machine, Sections II and IV]

REFERENCES

[1] G. B. de Pádua and W. Shang, "Studying the relationship between exception handling practices and post-release defects," in *Proc. 15th Int. Conf. Mining Software Repositories (MSR)*, 2018, doi: 10.1145/3196398.3196435.

[2] G. B. de Pádua and W. Shang, "Studying the prevalence of exception handling anti-patterns," in *Proc. 25th Int. Conf. Program Comprehension (ICPC)*, 2017.

[3] G. B. de Pádua and W. Shang, "Revisiting exception handling practices with exception flow analysis," in *Proc. 17th Int. Working Conf. Source Code Analysis and Manipulation (SCAM)*, 2017.

[4] J. Jiarpakdee, C. Tantithamthavorn, and A. E. Hassan, "The impact of correlated metrics on the interpretation of defect models," *IEEE Trans. Softw. Eng.*, vol. 47, no. 2, pp. 320–331, 2021.

[5] M. Shepperd, D. Bowes, and T. Hall, "Researcher bias: The use of machine learning in software defect prediction," *IEEE Trans. Softw. Eng.*, vol. 40, no. 6, pp. 603–616, 2014.

[6] C. Tantithamthavorn, S. McIntosh, A. E. Hassan, and K. Matsumoto, "An empirical comparison of model validation techniques for defect prediction models," *IEEE Trans. Softw. Eng.*, vol. 43, no. 1, pp. 1–18, 2017.

[7] F. J. Shull, J. C. Carver, S. Vegas, and N. Juristo, "The role of replications in empirical software engineering," *Empirical Softw. Eng.*, vol. 13, no. 2, pp. 211–218, 2008.

[8] F. E. Harrell, *Regression Modeling Strategies*, 2nd ed. Springer, 2015.